

Paso de Parámetros y Gestión de Memoria en C++

Asignatura: Programación Avanzada y Algoritmos (LIIS2105)

Universidad: Universidad Iberoamericana Puebla

1. Objetivo de la Práctica

El objetivo de esta sesión práctica es que los estudiantes analicen, implementen y contrasten experimentalmente el comportamiento en memoria, la seguridad de la información y la eficiencia de rendimiento de los tres mecanismos de paso de parámetros en C++: **paso por valor**, **paso por referencia**, y **paso por referencia constante**. A través de la simulación de un sistema de transacciones bancarias, los alumnos visualizarán mediante consola cuándo se invocan los constructores de copia y cómo se gestionan los efectos secundarios de los datos.

2. Fundamento Teórico

En la Programación Orientada a Objetos en C++, la forma en que pasamos objetos a las funciones y métodos tiene implicaciones críticas en dos áreas principales: la integridad de los datos (evitar efectos secundarios no deseados) y el consumo de recursos de la máquina (evitar copias innecesarias).

- **Paso por Valor (Pass-by-Value):** Se realiza una copia bit a bit del objeto original en una variable temporal albergada en el stack de la función. Esto invoca implícitamente al *constructor de copia* de la clase. Las modificaciones locales no afectan al objeto emisor, pero representa un alto costo de rendimiento para objetos grandes.
- **Paso por Referencia (Pass-by-Reference):** El parámetro se convierte en un alias directo (mismo espacio físico de memoria) de la variable original. No hay copias ni llamadas a constructores especiales, y cualquier cambio realizado dentro del método se refleja de inmediato en el objeto emisor externo.
- **Paso por Referencia Constante (const Tipo&):** Representa el estándar de oro para objetos en C++. Combina la eficiencia del paso por referencia (se comparte la dirección, sin llamadas al constructor de copia) con la seguridad del paso por valor (el compilador bloquea cualquier intento de modificar los atributos del objeto o de llamar a métodos que no estén marcados como const).

3. Código Fuente Base de Estudio

A continuación se presenta el código de la clase Cuenta. El programa ha sido diseñado con un constructor de copia explícito que emite un mensaje en consola. Esto te permitirá visualizar con exactitud cuándo el compilador de C++ duplica información en memoria.

Parte 1: Declaración de la Clase Cuenta

```
#include <iostream>
#include <string>
class Cuenta {
private:
    std::string titular;
    std::string numeroCuenta;
    double saldo;
public:
    // Constructor parametrizado con lista de inicializacion
    Cuenta(std::string titular_val, std::string num_val, double saldo_val)
        : titular(titular_val), numeroCuenta(num_val), saldo(saldo_val) {}
    // Constructor de copia explicito para evidenciar la duplicacion en memoria
    Cuenta(const Cuenta& otra)
        : titular(otra.titular), numeroCuenta(otra.numeroCuenta), saldo(otra.saldo) {
        std::cout << "    >>> [ALERTA] Constructor de copia invocado para la cuenta de "
            << titular << std::endl;
    }
    // Metodos constantes de acceso (Solo lectura)
    std::string obtenerTitular() const { return titular; }
    std::string obtenerNumero() const { return numeroCuenta; }
    double obtenerSaldo() const { return saldo; }
    // Metodos modificadores
    void retirar(double monto) { saldo -= monto; }
    void depositar(double monto) { saldo += monto; }
};
```

Parte 2: Definición de las Funciones de Prueba

```
// 1. PASO POR VALOR (Demuestra el aislamiento de datos y costo de copia)
void calcularProyeccionInteres(Cuenta c, float tasa) {
    std::cout << "\n[Funcion Proyeccion] Iniciando calculo..." << std::endl;
    double interes = c.obtenerSaldo() * tasa;
    c.depositar(interres);
    std::cout << "[Funcion Proyeccion] Saldo proyectado en COPIA temporal: $" << c.obtenerSaldo() << std::endl;
    std::cout << "[Funcion Proyeccion] Finalizando funcion (la copia se destruye)." << std::endl;
}

// 2. PASO POR REFERENCIA (Demuestra la persistencia de efectos secundarios)
void procesarTransferencia(Cuenta& origen, Cuenta& destino, double monto) {
    std::cout << "\n[Funcion Transferencia] Procesando transaccion de $" << monto << "..." << std::endl;
    if (origen.obtenerSaldo() >= monto) {
        origen.retirar(monto);
        destino.depositar(monto);
        std::cout << "[Funcion Transferencia] Exito. Fondos transferidos de manera real." << std::endl;
    } else {
        std::cout << "[Funcion Transferencia] Error: Fondos insuficientes." << std::endl;
    }
}

// 3. PASO POR REFERENCIA CONSTANTE (Demuestra eficiencia extrema + seguridad)
void imprimirAuditoria(const Cuenta& c) {
    std::cout << "\n[Funcion Auditoria] Impresion de seguridad de la cuenta:" << std::endl;
    std::cout << "  - Titular: " << c.obtenerTitular() << std::endl;
    std::cout << "  - Numero de Cuenta: " << c.obtenerNumero() << std::endl;
    std::cout << "  - Saldo Auditado: $" << c.obtenerSaldo() << std::endl;

    // TRAMPA PEDAGOGICA: Descomentar la linea de abajo causara error de compilacion
    // c.depositar(100);
}
```

Parte 3: Función Principal (main)

```
int main() {
    std::cout << "=== Inicializando cuentas bancarias ===" << std::endl;
    Cuenta cuentaJuan("Juan Perez", "123-456", 1000.0);
    Cuenta cuentaMaria("Maria Gomez", "789-012", 500.0);

    // CASO 1: Demostracion de Paso por Valor
    std::cout << "\n--- Ejecutando Caso 1 (Paso por Valor) ---" << std::endl;
    calcularProyeccionInteres(cuentaJuan, 0.05f);
    std::cout << "Saldo final de Juan en main(): $" << cuentaJuan.obtenerSaldo() << std::endl;

    // CASO 2: Demostracion de Paso por Referencia
    std::cout << "\n--- Ejecutando Caso 2 (Paso por Referencia) ---" << std::endl;
    procesarTransferencia(cuentaJuan, cuentaMaria, 300.0);
    std::cout << "Saldo de Juan en main(): $" << cuentaJuan.obtenerSaldo() << std::endl;
    std::cout << "Saldo de Maria en main(): $" << cuentaMaria.obtenerSaldo() << std::endl;

    // CASO 3: Demostracion de Paso por Referencia Constante
    std::cout << "\n--- Ejecutando Caso 3 (Paso por Referencia Constante) ---" << std::endl;
    imprimirAuditoria(cuentaJuan);

    return 0;
}
```

4. Guía de Ejecución y Ejercicio de Reflexión

Paso 1: Copia el código anterior en tu entorno de desarrollo local (como VS Code o CLion), compíllalo y ejecútalo. Analiza con detenimiento la salida generada por la terminal.

Salida Esperada en Consola:

```
=== Inicializando cuentas bancarias ===
--- Ejecutando Caso 1 (Paso por Valor) ---
>>> [ALERTA] Constructor de copia invocado para la cuenta de Juan Perez
[Funcion Proyeccion] Iniciando calculo...
[Funcion Proyeccion] Saldo proyectado en COPIA temporal: $1050
[Funcion Proyeccion] Finalizando funcion (la copia se destruye).
Saldo final de Juan en main(): $1000
--- Ejecutando Caso 2 (Paso por Referencia) ---
[Funcion Transferencia] Procesando transaccion de $300...
[Funcion Transferencia] Exito. Fondos transferidos de manera real.
Saldo de Juan en main(): $700
Saldo de Maria en main(): $800
--- Ejecutando Caso 3 (Paso por Referencia Constante) ---
[Funcion Auditoria] Impresion de seguridad de la cuenta:
- Titular: Juan Perez
- Numero de Cuenta: 123-456
- Saldo Auditado: $700
```

Paso 2: Cuestionario de Control

Responde de manera individual y con rigor técnico las siguientes preguntas basadas en tu observación del comportamiento de la memoria del programa:

- ? **Pregunta 1:** ¿Por qué se disparó el mensaje de [ALERTA] en el Caso 1, pero no en el Caso 2 ni en el Caso 3? Explica la implicación del constructor de copia en términos de sobrecarga de CPU y memoria.
- ? **Pregunta 2:** En el Caso 1, a pesar de que en la función calcularProyeccionInteres se ejecutó el método depositar con éxito, ¿por qué el saldo de Juan en el programa principal siguió reportando \$1000 al finalizar?
- ? **Pregunta 3:** Descomenta la línea `c.depositar(100);` en la función `imprimirAuditoria`. Intenta compilar el programa. ¿Qué error de compilación obtienes? ¿Por qué la palabra clave `const` restringe esta acción y cómo protege el encapsulamiento de la clase?

Sugerencia Pedagógica para el Docente: Utilice este ejercicio como punto de partida para explicar por qué las referencias constantes (`const T&`) son el estándar de oro en el desarrollo profesional de videojuegos, simuladores y sistemas embebidos, donde la duplicación de objetos pesados en el stack degrada de forma acumulativa el rendimiento de la aplicación.

5. Rúbrica de Evaluación

Para acreditar este laboratorio, deberás entregar un reporte en formato PDF que incluya la captura de la compilación, el cuestionario de control resuelto y conclusiones analíticas. La evaluación se registrará bajo los siguientes criterios:

Criterio de Evaluación	Descripción Detallada	Puntaje
Conocimiento Técnico	Identifica con precisión cuándo se invoca el constructor de copia y explica correctamente la diferencia de ámbitos en el stack para cada caso de prueba.	30%
Implementación Práctica	El código fuente compila sin advertencias ni errores. Demuestra un uso correcto de las listas de inicialización y modificadores de acceso.	30%
Pensamiento Crítico	Resuelve el cuestionario de control justificando técnicamente sus afirmaciones con base en la teoría de gestión de memoria y protección de datos.	30%
Presentación y Formato	Entrega un reporte ordenado, con ortografía impecable, capturas claras de la ejecución y en el tiempo establecido por la plataforma Moodle.	10%

6. Referencias Bibliográficas (Estilo APA)

- Ceballos Sierra, F. J. (2015). *Programación orientada a objetos con C++* (4.ª ed.). RA-MA Editorial.
- Finochietto, J. M. (2010). *Programación Orientada a Objetos en C++*. Universidad Nacional de Córdoba (UNC-CONICET).
- García Peñalvo, F. J., & Hernández Simón, J. A. (2005). *2 - Clases y objetos en C++*. Departamento de Informática y Automática, Universidad de Salamanca.
- Garrido Carrillo, A. (2016). *Prácticas con C++: metodología de la programación* (2.ª ed.). Editorial Universidad de Granada.
- González Pérez, A. (2025). *El gran libro de programación en C++* (1.ª ed.). Marcombo.
- Google. (2026). *Gemini Notebook [Asistente de Inteligencia Artificial]*. <https://notebook.google.com/>